UNIVERSITÉ DU QUÉBEC EN OUTAOUAIS

Département d'informatique et d'ingénierie

INF6303 – Techniques d'analyse des mégadonnées

Devoir 1

Pipeline analytique polyglotte

Hive · MongoDB · Cassandra

PROFESSEUR

Prof. Ndah Daniel Yapi, Ph.D.

GROUPE 6

Agouanet, Franklin Platini
Kanjiep Kepgnou, Mikael
El Hajjaji, Sabra
Kapawa, Gaston Raphaël
Deberne, Hugo Bernard Denis

Automne 2026 · Remis le 26 octobre 2026

Sommaire

Mandat et démarche **3**

Section 3 – Démarrage et contrôle du volume **4**

Nombre de fichiers et volume HDFS 4

Pourquoi 8 fichiers gzip donnent plus de parallélisme qu'un seul 4

Écart entre le volume compressé et le nombre logique de lignes 4

Partie A – Hive/HiveQL **6**

A1. Tables et qualité 6

A2. Analyse distribuée 8

EXPLAIN de la requête du top 10 10

A3. Sortie transmise à MongoDB 11

Partie B – MongoDB **16**

1. Objet et périmètre 16

2. Architecture et flux de données 16

3. B1 — Importation documentaire et indexation 16

4. B2 — Enrichissement des profils 17

5. Analyse des performances : index composé et parcours naturel 17

6. B3 — Sélection et exportation de l'audience 18

7. Contrôles de qualité et reproductibilité 19

8. Difficultés rencontrées et impacts 19

9. Limites, intégration et conclusion 20

Partie C – Cassandra **21**

C1. Modèle orienté requête 21

C2. Chargement 23

C3. Requêtes de service 23

Partie D – Validation et architecture **26**

D1. Traçabilité 26

D2. Architecture 30

Conclusion **33**

Annexes – preuves d'exécution **34**

Annexe 1 – Sortie de 04_validation_pipeline.py (output/log_D.txt) 34

Annexe 2 – Sortie de 03_cassandra_publication.py (output/log_C.txt) 35

Mandat et démarche

Une plateforme de diffusion veut transformer un historique massif d'événements en une audience opérationnelle. Le pipeline combine trois technologies, et chacune a un rôle précis :

Étape	Technologie	Résultat transmis
Analyse massive	Hive sur HDFS	`segments_utilisateurs.csv`
Enrichissement documentaire	MongoDB	`audience_cible.jsonl`
Publication à faible latence	Cassandra	Table interrogeable par pays et segment

Question directrice. Comment articuler un stockage analytique distribué, un modèle documentaire flexible et une base orientée requête sans confondre leurs responsabilités ?

Le rapport suit l'ordre du pipeline. Chaque partie présente les commandes exécutées, les résultats obtenus, les contrôles de cohérence et la justification des choix techniques. La procédure complète de reproduction, à partir d'un environnement Docker remis à zéro, et la répartition du travail se trouvent dans `README_EQUIPE.md`.

Principes respectés de bout en bout :

- Seules les données fournies sont utilisées, et aucun identifiant n'est fabriqué entre les étapes.
- `user_id` est la clé d'interconnexion des trois systèmes.
- Le million d'événements reste dans HDFS. Seuls les agrégats par utilisateur (50 000 lignes) passent à MongoDB, puis seule l'audience ciblée (34 855 documents) passe à Cassandra.
- Chaque fichier de passage est consommé par la technologie suivante, et ce passage est vérifié par des comptages et des comparaisons de valeurs (partie D1).

Fichier	Rôle
`workspace/01_hive_analyse.sql`	Partie A : tables externes, analyses, segmentation et export
`workspace/02_mongodb_enrichissement.py`	Partie B : import, index, enrichissement et export
`workspace/03_cassandra_publication.py`	Partie C : modèle, chargement et requêtes de service
`workspace/04_validation_pipeline.py`	Partie D1 : comptages et traçabilité de bout en bout
`workspace/output/segments_utilisateurs.csv`	Fichier de passage Hive → MongoDB
`workspace/output/audience_cible.jsonl`	Fichier de passage MongoDB → Cassandra
`workspace/output/log_A.txt`, `log_C.txt`, `log_D.txt`	Preuves d'exécution (parties A, C et D1)

Section 3 – Démarrage et contrôle du volume

Nombre de fichiers et volume HDFS

```
$ hdfs dfs -ls /inf6303/devoir1/evenements
Found 8 items
-rw-r--r--   1 jovyan supergroup    3064906 ... evenements_part_01.csv.gz
...                                (8 fichiers de 3 064 220 à 3 065 291 octets)
$ hdfs dfs -du -s -h /inf6303/devoir1/evenements
23.4 M  23.4 M  /inf6303/devoir1/evenements
$ hdfs dfs -count -h /inf6303/devoir1/evenements
           1            8             23.4 M /inf6303/devoir1/evenements
```

Mesure	Valeur
Fichiers dans `/inf6303/devoir1/evenements`	**8**
Volume HDFS (compressé)	**24 517 828 octets (23,4 Mio)**, environ 2,9 Mio par fichier
Espace disque consommé (2e colonne de du)	23,4 Mio (facteur de réplication 1, un seul DataNode)
Taille de bloc HDFS (`dfs.blocksize`)	128 Mio
Blocs (`hdfs fsck`)	8 : **un bloc par fichier**
Lignes physiques (`zcat | wc -l`)	1 000 008 = 1 000 000 événements + 8 en-têtes
Volume décompressé	89 405 327 octets (85,3 Mio)

Le catalogue est déposé dans `/inf6303/devoir1/catalogue` (1 fichier, 83 080 octets).

Pourquoi 8 fichiers gzip donnent plus de parallélisme qu'un seul

- Dans MapReduce, une tâche *map* traite un *split*. Pour un fichier texte non compressé, Hadoop coupe le fichier en splits, en général un par bloc, et chaque mapper commence à lire au milieu du fichier, à la première fin de ligne.
- **gzip n'est pas découpable.** Un flux deflate ne peut être décodé qu'à partir de son début : on ne peut pas commencer la lecture au milieu du fichier. Hadoop le sait (`GzipCodec` n'est pas un `SplittableCompressionCodec`) et crée **un seul split par fichier .gz**, quelle que soit sa taille ou son nombre de blocs.
- Avec un seul fichier gzip, un seul mapper lirait donc le million de lignes, même sur un cluster de 100 nœuds. S'il dépassait 128 Mio, ce mapper devrait en plus aller lire les blocs suivants sur d'autres nœuds (perte de localité des données).
- Avec 8 fichiers, on obtient **8 splits, donc 8 mappers** qui lisent chacun 125 000 lignes en parallèle. On le constate dans `log_A.txt` : chaque requête sur `evenements_massifs` affiche `number of splits:8`. La jointure avec le catalogue affiche `number of splits:9` (8 + 1).
- Le découpage en amont, au moment de la production des fichiers, remplace le découpage que gzip interdit. Le degré de parallélisme maximal de la lecture est le nombre de fichiers. Des formats découpables (bzip2, ou ORC/Parquet compressés par blocs internes) lèveraient cette limite.
- Remarque sur cet environnement : YARN est instable (voir le README), donc les jobs tournent en MapReduce local. Les 8 tâches map s'exécutent alors dans la JVM de HiveServer2. Le découpage en 8 splits est le même que sur un cluster, mais le gain réel en temps n'apparaît qu'avec plusieurs nœuds.

Écart entre le volume compressé et le nombre logique de lignes

- HDFS mesure des **octets stockés**, pas des lignes. `du` donne 23,4 Mio, qui sont des octets compressés. Ce chiffre ne permet pas de déduire le nombre d'événements.

- Le contenu logique est 3,65 fois plus gros : 89,4 Mo décompressés pour 24,5 Mo stockés. Une ligne CSV fait en moyenne 89 octets décompressés, mais seulement environ 24,5 octets compressés. gzip est très efficace ici, car les colonnes répètent peu de valeurs (8 pays, 5 types, 4 appareils, 4 qualités) et les identifiants ont des préfixes communs.
- Le nombre logique de lignes n'est connu qu'après décompression et lecture : `zcat | wc -l` donne 1 000 008 lignes physiques. Il faut retirer les 8 en-têtes, un par fichier, ignorés par `skip.header.line.count`, pour obtenir **1 000 000 événements**, confirmé par `COUNT(*)` en A1.
- Conséquence pour Hive : le planificateur estime les volumes à partir des tailles de fichiers compressés. L'`EXPLAIN` ci-dessous annonce environ 224 522 lignes pour la table des événements, alors qu'il y en a 1 000 000. Sans `ANALYZE TABLE`, les statistiques (`Column stats: NONE`) sous-estiment fortement la donnée réelle.

Partie A – Hive/HiveQL

Script : `workspace/01_hive_analyse.sql` · Sortie complète : `workspace/output/log_A.txt`

```
docker compose --profile hadoop up -d --build
# Terminal JupyterLab
bash /home/jovyan/work/00_deposer_dans_hdfs.sh
beeline -u jdbc:hive2://hive:10000/default -n jovyan \
  -f /home/jovyan/work/01_hive_analyse.sql
hdfs dfs -getmerge /inf6303/devoir1/sorties/segments /tmp/segments_utilisateurs.csv
cp /tmp/segments_utilisateurs.csv /home/jovyan/work/output/segments_utilisateurs.csv
```

Entrées : `data/evenements/*.csv.gz` (8 fichiers) et `data/catalogue_contenus.csv`, déposés dans HDFS. Le script est rejouable : il recrée ses tables externes (`DROP` ne supprime que la définition, jamais les fichiers HDFS) et réécrit le répertoire de sortie (`INSERT OVERWRITE DIRECTORY`). Les résultats A2 sont **comparés dans la requête elle-même** aux valeurs de `data/MANIFESTE_DONNEES.json` (colonne `controle`) : tous les contrôles valent `OK`.

A1. Tables et qualité

Tables externes typées

```
CREATE EXTERNAL TABLE evenements_massifs (          -- Franklin, inchangée
  event_id STRING, user_id STRING, content_id STRING, event_type STRING,
  event_ts STRING, country STRING, device STRING, watch_seconds INT,
  quality STRING, session_id STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE
LOCATION '/inf6303/devoir1/evenements'
TBLPROPERTIES ('skip.header.line.count'='1');

CREATE EXTERNAL TABLE catalogue_contenus (
  content_id   STRING,   -- clé de jointure avec evenements_massifs.content_id
  title        STRING,
  genre        STRING,
  release_year INT,
  duration_sec INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE
LOCATION '/inf6303/devoir1/catalogue'
TBLPROPERTIES ('skip.header.line.count'='1');
```

- **EXTERNAL** : Hive ne possède pas les fichiers. `DROP TABLE` ne supprime que la définition, et les données déposées restent dans HDFS. Le schéma est appliqué à la lecture.
- **Typage** : les mesures sont en `INT` (`watch_seconds`, `duration_sec`, `release_year`). Une valeur non numérique devient `NULL` à la lecture, ce qui la rend détectable (contrôles ci-dessous). L'ordre des colonnes suit l'en-tête réel des fichiers.
- **Fins de ligne du catalogue** : `catalogue_contenus.csv` est fourni en CRLF, donc le `\r` reste collé à `duration_sec`. Ce n'est pas un problème : Hive convertit bien la valeur en `INT` (0 `NULL` sur 2 000, contrôle ci-dessous). Le fichier source n'a pas été modifié, il doit rester conforme à `SHA256SUMS.txt`.
- **Paramètres d'exécution** en tête du script (voir README) : MapReduce local, pas de map-join automatique (mémoire), et `HiveInputFormat`. Avec le format par défaut `CombineHiveInputFormat`, les 8 en-têtes étaient comptés comme des données (1 000 008).

Volumes confirmés

Contrôle	Obtenu	Attendu
`COUNT(*) evenements_massifs`	**1 000 000**	1 000 000

Contrôle	Obtenu	Attendu
`COUNT(*) catalogue_contenus`	**2 000**	2 000
`COUNT(DISTINCT content_id)` catalogue	2 000	2 000

Contrôles de qualité et explication

Contrôle	Règle	Résultat	Pourquoi ce contrôle
`user_id` invalide	`NULL` ou vide	0	`user_id` est la clé d'interconnexion avec MongoDB et Cassandra. Une valeur vide créerait un faux utilisateur en A3.
Format `user_id`	`RLIKE '^u[0-9]{6}$'`	0	Une valeur tronquée ou un décalage de colonnes (virgule dans un champ) passe le test « non vide » mais pas celui du format.
Format `event_id`	`RLIKE '^e[0-9]{9}$'`	0	Même raison, pour la clé de l'événement.
`event_id` en double	`COUNT(*) - COUNT(DISTINCT event_id)`	0	Un doublon (fichier déposé deux fois, par exemple) gonflerait les comptages et les durées.
Format `content_id`	`RLIKE '^c[0-9]{5}$'`	0	`content_id` est la clé de jointure du top 10.
`content_id` absent du catalogue	`LEFT JOIN, c.content_id IS NULL`	0	La jointure interne du top 10 perdrait ces événements sans le signaler. Les 2 000 contenus du catalogue sont tous utilisés.
Unicité de la clé du catalogue	`COUNT(DISTINCT content_id) = 2 000`	OK	Un `content_id` en double dans la dimension dupliquerait des lignes à la jointure et fausserait les sommes.
Durée `NULL` ou non numérique	`watch_seconds IS NULL`	0	Le typage `INT` transforme une valeur illisible en `NULL`. `SUM` l'ignorerait en silence.
Durée négative	`watch_seconds < 0`	0	Une durée négative est impossible et réduirait la durée totale d'un utilisateur.
Durée nulle (0 s)	`watch_seconds = 0`	2 566	Ces durées sont **valides** : toutes concernent des événements sans lecture (pause, recherche, abandon, favori). Il y a 0 lecture à 0 s.
Plage des durées	`MIN` et `MAX`	0 à 7 200 s	Aucune valeur aberrante

Contrôle	Règle	Résultat	Pourquoi ce contrôle
			(au plus 2 h par événement).
Durée > durée du contenu	`watch_seconds > duration_sec`	224 988	Signalée mais **non exclue** (voir ci-dessous).
Catalogue	`duration_sec` NULL ou ≤ 0, `release_year` NULL	0 / 0	Durées de 308 à 8 395 s, années de 1990 à 2026.

Durées par type d'événement (elles expliquent les 2 566 durées nulles) :

event_type	événements	min (s)	max (s)	moyenne (s)
lecture	549 714	20	7 200	3 611,1
pause	139 884	0	180	90,0
recherche	119 987	0	180	90,0
abandon	110 611	0	180	89,8
ajout_favori	79 804	0	180	90,0

Durée regardée supérieure à la durée du contenu (224 988 événements, 22,5 %). C'est un contrôle de cohérence métier, pas une erreur de format. On le signale sans exclure ces lignes, pour trois raisons :

- `watch_seconds` peut cumuler le temps d'une session (revisionnage, retour en arrière).
- Les exclure modifierait la sortie A3 transmise à MongoDB, qui doit couvrir tous les événements.
- En les gardant, le top 10 et les durées totales correspondent exactement au manifeste : ses valeurs attendues ont donc été calculées sur l'ensemble des événements.

Bilan A1 : aucune ligne invalide. Le filtre de la requête A3 (`user_id` non vide, durée non `NULL` et ≥ 0) ne retire rien : la somme des `event_count` de la sortie vaut bien 1 000 000.

A2. Analyse distribuée

Répartition des événements par pays et par type

Un seul passage sur le million de lignes produit le croisement pays × type (40 lignes, table temporaire `repartition_pays_type`). Les deux répartitions simples sont ensuite dérivées de ces 40 lignes au lieu de relire le million d'événements deux fois.

Par pays, comparée à `events_by_country` du manifeste :

Pays	Événements	%	Manifeste	Contrôle
Canada	311 247	31,12	311 247	OK
Senegal	132 617	13,26	132 617	OK
Cote_d_Ivoire	119 916	11,99	119 916	OK
France	118 790	11,88	118 790	OK
Maroc	98 758	9,88	98 758	OK
Cameroun	89 530	8,95	89 530	OK
Belgique	69 962	7,00	69 962	OK
Tunisie	59 180	5,92	59 180	OK

Par type, comparée à `events_by_type` :

Type	Événements	%	Manifeste	Contrôle
lecture	549 714	54,97	549 714	OK
pause	139 884	13,99	139 884	OK
recherche	119 987	12,00	119 987	OK
abandon	110 611	11,06	110 611	OK
ajout_favori	79 804	7,98	79 804	OK

Croisement pays × type :

Pays	lecture	pause	recherche	abandon	ajout_favori
Belgique	38 404	9 814	8 424	7 716	5 604
Cameroun	49 451	12 475	10 779	9 781	7 044
Canada	170 961	43 623	37 242	34 537	24 884
Cote_d_Ivoire	65 889	16 723	14 413	13 356	9 535
France	65 322	16 582	14 231	13 177	9 478
Maroc	54 558	13 786	11 692	10 798	7 924
Senegal	72 561	18 608	16 146	14 722	10 580
Tunisie	32 568	8 273	7 060	6 524	4 755

Lecture : le Canada représente près du tiers des événements, ce qui explique la partition chaude Canada/actif discutée en partie C. Le mélange des types est presque identique d'un pays à l'autre (environ 55 % de lectures partout) : les pays diffèrent par leur volume, pas par leur usage.

Top 10 des contenus par durée totale (après jointure avec le catalogue)

```
SELECT e.content_id, c.title, c.genre, c.duration_sec,
       SUM(e.watch_seconds) AS duree_totale_sec, COUNT(*) AS nb_evenements
FROM evenements_massifs e
JOIN catalogue_contenus c ON e.content_id = c.content_id
GROUP BY e.content_id, c.title, c.genre, c.duration_sec
ORDER BY duree_totale_sec DESC
LIMIT 10;
```

#	content_id	titre	genre	durée totale (s)	événements	Manifeste	Contrôle
1	c00545	Contenu_00545	jeunesse	1 226 712	564	c00545 · 1 226 712	OK
2	c01844	Contenu_01844	actualite	1 221 503	517	c01844 · 1 221 503	OK
3	c00686	Contenu_00686	comedie	1 216 833	526	c00686 · 1 216 833	OK
4	c00277	Contenu_00277	sport	1 210 971	568	c00277 · 1 210 971	OK
5	c00553	Contenu_00553	science_fiction	1 204 604	529	c00553 · 1 204 604	OK
6	c01825	Contenu_01825	drame	1 201 793	524	c01825 · 1 201 793	OK
7	c00882	Contenu_00882	drame	1 200 490	510	c00882 · 1 200 490	OK
8	c00912	Contenu_0	comedie	1 199 344	494	c00912 · 1	OK

#	content_id	titre	genre	durée totale (s)	événements	Manifeste	Contrôle
		0912				199 344	
9	c01263	Contenu_01263	jeunesse	1 198 489	567	c01263 · 1 198 489	OK
10	c01769	Contenu_01769	jeunesse	1 198 083	519	c01769 · 1 198 083	OK

Le rang, le `content_id` et la durée totale sont identiques au manifeste pour les 10 lignes. La jointure apporte le titre et le genre, qui n'existent pas dans les événements. Les écarts sont faibles (1,23 M contre 1,20 M de secondes) : aucun contenu ne domine nettement, et trois des dix sont des contenus jeunesse.

Durée moyenne par appareil et qualité

`duree_moy_sec` porte sur tous les événements. `duree_moy_lecture_sec` ne garde que les lectures, car les autres types ne durent que de 0 à 180 s et tirent la moyenne vers le bas.

Appareil	Qualité	Événements	Durée moy. (s)	Durée moy. lectures (s)
mobile	4K	52 292	2 028,6	3 606,9
mobile	HD	155 778	2 026,3	3 623,4
mobile	FHD	143 906	2 023,3	3 609,5
mobile	SD	47 933	2 012,9	3 595,9
ordinateur	4K	27 203	2 032,4	3 596,4
ordinateur	FHD	75 425	2 028,9	3 608,2
ordinateur	HD	82 003	2 022,4	3 597,7
ordinateur	SD	25 236	2 014,5	3 599,4
tablette	SD	9 767	2 065,3	3 642,0
tablette	4K	10 327	2 025,3	3 605,2
tablette	FHD	28 825	2 019,9	3 634,5
tablette	HD	31 232	2 010,6	3 591,6
television	FHD	111 535	2 034,5	3 621,5
television	SD	37 343	2 032,1	3 618,5
television	HD	121 069	2 028,1	3 608,8
television	4K	40 126	2 011,3	3 605,9

Lecture des résultats :

- Les moyennes sont très proches : de 2 010 à 2 065 s sur tous les événements, et de 3 592 à 3 642 s sur les lectures. L'appareil et la qualité ne changent pratiquement pas la durée d'une session dans ce jeu de données.
- Ce qui varie, c'est le **volume** : le mobile (399 909 événements) et la télévision (310 073) dominent, et les qualités HD et FHD sont les plus fréquentes.
- La plus forte moyenne (tablette/SD) est aussi le plus petit groupe (9 767 événements), donc le plus sensible au hasard. Aucun manifeste ne donne de valeur attendue pour cette analyse.

EXPLAIN de la requête du top 10

Plan complet dans `log_A.txt`. Avec `hive.auto.convert.join=false`, Hive produit **3 jobs MapReduce en chaîne** et une étape de lecture finale :

Étape	Map	Shuffle (clé de partition)	Reduce
Stage-1	`TableScan e` (événements) et `TableScan c` (catalogue), filtre `content_id is not null`, projection	**`Reduce Output Operator` sur `content_id`, pour les deux tables**	`Join Operator` (`Inner Join 0 to 1`), puis `Group By` partiel (`mode: hash`)
Stage-2	relecture du résultat intermédiaire	`content_id, title, genre, duration_sec`	`Group By` final (`mode: mergepartial`)
Stage-3	relecture, `TopN Hash Memory Usage: 0.1`	**tri global** `sort order: -` sur `duree_totale_sec`, sans colonne de partition, donc un seul reducer	`Limit 10`
Stage-0	`Fetch Operator`, `limit: 10`	–	–

Opération coûteuse : la jointure en phase reduce (shuffle) du Stage-1

- C'est une **jointure côté reduce** (*common join*). Pour réunir les lignes de même `content_id`, Hive envoie **les deux tables** dans le shuffle (`Reduce Output Operator`, clé `content_id`). Les 1 000 000 événements sont donc sérialisés, triés, transférés et fusionnés, alors que la table jointe ne fait que 2 000 lignes (83 Ko). C'est l'étape qui déplace le plus de données. C'est aussi le seul passage sur le million de lignes : les étapes suivantes ne traitent qu'environ 2 000 groupes.
- Sur un vrai cluster, ce shuffle traverse le réseau. Le reducer qui reçoit un `content_id` populaire reçoit aussi toutes ses lignes (risque de déséquilibre).
- **Amélioration** : une **map-join** (*broadcast*). Le catalogue de 83 Ko tient en mémoire. Avec `hive.auto.convert.join=true`, Hive le diffuse à chaque mapper et fait la jointure côté map : le Stage-1 n'a plus de shuffle d'événements. Elle est désactivée ici uniquement parce que la tâche locale qui prépare la table de hachage dépasse la mémoire du conteneur Hive (`-Xmx1G`, voir README). Autre option : agréger d'abord les événements par `content_id`, puis joindre les 2 000 lignes agrégées.

Deuxième point coûteux : le tri global (Stage-3)

- `ORDER BY` impose un ordre total : le plan n'a **pas de colonne de partition**, et tout passe par **un seul reducer**. Sur un grand résultat, c'est un goulot d'étranglement qu'on ne peut pas paralléliser.
- Ici, le coût est limité pour deux raisons : le tri ne porte que sur environ 2 000 lignes agrégées, et Hive applique le `LIMIT` dès la phase map (`TopN Hash Memory Usage: 0.1`). Chaque mapper n'envoie que ses 10 meilleures lignes au reducer unique.

Statistiques du plan

Les estimations (`Num rows: 224522` pour les événements, 1675 pour le catalogue) sont fausses, car `Column stats: NONE` : Hive les déduit de la taille des fichiers, compressés pour les événements. Un `ANALYZE TABLE ... COMPUTE STATISTICS` donnerait de vraies cardinalités. Le choix automatique d'une map-join repose justement sur ces tailles estimées.

A3. Sortie transmise à MongoDB

Objectif. Calculer, pour chaque `user_id`, le pays, le nombre d'événements et la durée totale de visionnage, puis attribuer un segment selon les seuils du complément du 6 octobre 2026. Le résultat est écrit dans HDFS, puis rapatrié dans `output/segments_utilisateurs.csv` pour MongoDB.

Script : `01_hive_analyse.sql`, gabarit du professeur, zones **TODO A1** (table `evenements_massifs`) et **TODO A4** (segmentation et export) · Base Hive : `inf6303_devoir1` · Journal d'exécution : `output/log_A.txt`

1. Préparation : table externe et paramètres d'exécution

Commande (début du gabarit, puis zone TODO A1)

```
CREATE DATABASE IF NOT EXISTS inf6303_devoir1;   -- imposé par le gabarit
USE inf6303_devoir1;

SET mapreduce.framework.name=local;
SET hive.exec.mode.local.auto=false;
SET hive.auto.convert.join=false;
SET hive.input.format=org.apache.hadoop.hive.ql.io.HiveInputFormat;

DROP TABLE IF EXISTS evenements_massifs;
CREATE EXTERNAL TABLE evenements_massifs (
  event_id STRING, user_id STRING, content_id STRING, event_type STRING,
  event_ts STRING, country STRING, device STRING, watch_seconds INT,
  quality STRING, session_id STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/inf6303/devoir1/evenements'
TBLPROPERTIES ('skip.header.line.count'='1');
```

Justification

- **Table externe** : Hive ne possède pas les fichiers. Un `DROP TABLE` supprime seulement la définition, jamais les données dans HDFS. On peut donc recréer la table sans risque pour garantir le bon schéma, alors qu'un `IF NOT EXISTS` aurait conservé sans aucun message une ancienne définition erronée.
- **Ordre des colonnes** : il a été vérifié sur l'en-tête réel des fichiers (`gzip -cd ... | head -3`). Un ordre incorrect ne produit pas d'erreur : Hive remplit les colonnes avec des valeurs `NULL`.
- `watch_seconds INT` : une valeur non numérique devient `NULL`, ce qui permet de la détecter au contrôle 2a.
- **Paramètres d'exécution** : MapReduce s'exécute en mode local, parce que les services YARN (ResourceManager et NodeManager) s'arrêtaient faute de mémoire. La jointure automatique par *map-join* est désactivée parce que sa tâche locale dépassait la mémoire du conteneur Hive. Les résultats sont identiques ; seul le lieu d'exécution change.

2. Contrôle du volume : un piège détecté et corrigé

Résultat

Exécution	`hive.input.format`	`COUNT(*)`	`watch_seconds` NULL
1re	par défaut (`CombineHiveInputFormat`)	**1 000 008**	8
2e	`HiveInputFormat`	**1 000 000**	0

Vérification. L'écart est de **8 lignes, exactement le nombre de fichiers**. Les 8 lignes en trop avaient toutes `watch_seconds = NULL`, parce que le texte `"watch_seconds"` de l'en-tête ne peut pas être converti en entier. Ce sont donc les **8 lignes d'en-tête**, lues comme si c'étaient des données.

Justification. Le format d'entrée par défaut regroupe plusieurs fichiers dans un même *split*. Dans ce cas, `skip.header.line.count` n'a pas été appliqué. `HiveInputFormat` traite chaque fichier séparément, et l'en-tête de chacun est bien ignoré. Sans ce contrôle, la segmentation n'aurait pas été faussée, puisque les lignes `NULL` étaient de toute façon exclues. En revanche, le volume annoncé dans le rapport aurait été faux.

3. Contrôles de qualité avant l'agrégation

Commande (2a et 2b)

```
-- 2a. Lignes invalides
SELECT
  SUM(CASE WHEN user_id IS NULL OR TRIM(user_id) = '' THEN 1 ELSE 0 END) AS
user_id_invalide,
  SUM(CASE WHEN watch_seconds IS NULL THEN 1 ELSE 0 END)               AS
duree_nulle_ou_non_numerique,
  SUM(CASE WHEN watch_seconds < 0     THEN 1 ELSE 0 END)               AS duree_negative
FROM evenements_massifs;

-- 2b. Utilisateurs associés à plusieurs pays
SELECT COUNT(*) AS nb_users_multi_pays
FROM (SELECT user_id FROM evenements_massifs
      GROUP BY user_id HAVING COUNT(DISTINCT country) > 1) t;
```

Résultat

Contrôle	Valeur
`user_id` vide ou NULL	0
Durée NULL ou non numérique	0
Durée négative	0
Utilisateurs avec plusieurs pays	**0**

Justification. Le filtre sur les lignes invalides est conservé dans la requête d'agrégation même s'il ne retire aucune ligne ici. Il rend le script robuste si les données changent. Le contrôle 2b justifie un **choix de conception** : comme chaque utilisateur a un seul pays, `MAX(country)` le récupère directement, dans la même agrégation. Une première version calculait le pays le plus fréquent avec une sous-requête et une jointure. Elle était inutilement coûteuse (deux passes et une jointure), et elle échouait par manque de mémoire.

4. Agrégation et segmentation

Commande

```
INSERT OVERWRITE DIRECTORY '/inf6303/devoir1/sorties/segments'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
SELECT
  user_id,
  MAX(country)       AS country,
  COUNT(*)           AS event_count,
  SUM(watch_seconds) AS total_watch_sec,
  CASE
    WHEN SUM(watch_seconds) >= 55000 THEN 'intensif'
    WHEN SUM(watch_seconds) >= 30000 THEN 'actif'
    ELSE 'occasionnel'
  END AS segment
FROM evenements_massifs
WHERE user_id IS NOT NULL AND TRIM(user_id) <> ''
  AND watch_seconds IS NOT NULL AND watch_seconds >= 0
GROUP BY user_id;
```

Justification

- La segmentation repose **uniquement** sur `SUM(watch_seconds)`, comme imposé par le complément. `event_count` est calculé et conservé, mais il n'intervient pas dans le segment.

- L'ordre des `WHEN` (du seuil le plus haut au plus bas) rend les conditions exclusives. Un utilisateur à 55 000 s est classé intensif, et non actif.
- Une seule passe MapReduce : 8 tâches *map* (une par fichier gzip), puis un *reduce* qui regroupe par `user_id`. L'exécution a pris **7,1 s**.

5. Vérification de la sortie

Une table externe `segments_sortie` est créée sur le dossier de sortie. Elle permet de vérifier le résultat **avec Hive lui-même**, avant de le transmettre.

Résultat (4a) : conservation des volumes

Mesure	Valeur	Attendu	
Lignes en sortie	50 000	≤ 50 000 profils	✅
`user_id` distincts	50 000	= nombre de lignes (aucun doublon)	✅
Somme des `event_count`	1 000 000	= événements valides	✅

Résultat (4b) : répartition et respect des seuils

Segment	Utilisateurs	Part	Min (s)	Max (s)	Seuil respecté
actif	30 901	61,8 %	30 003	54 999	[30 000 ; 55 000[✅
occasionnel	11 669	23,3 %	1 574	29 999	< 30 000 ✅
intensif	7 430	14,9 %	55 000	106 412	≥ 55 000 ✅
Total	**50 000**	100 %			

Vérification. Les bornes observées (29 999 / 30 003 et 54 999 / 55 000) montrent qu'aucun utilisateur n'est mal classé à la frontière d'un seuil. La somme des `event_count` est égale au nombre d'événements : l'agrégation n'a perdu ni dupliqué aucun événement.

Échantillon (4c)

```
u000001,Senegal,26,60487,intensif
u000002,Canada,17,26137,occasionnel
u000003,Cote_d_Ivoire,27,45954,actif
u000004,Canada,18,51017,actif
u000005,Canada,17,35253,actif
```

6. Rapatriement vers MongoDB

Commande fournie (terminal JupyterLab)

```
rm -f /home/jovyan/work/output/segments_utilisateurs.csv
hdfs dfs -getmerge /inf6303/devoir1/sorties/segments \
  /home/jovyan/work/output/segments_utilisateurs.csv
wc -l /home/jovyan/work/output/segments_utilisateurs.csv
```

Résultat

```
$ wc -l output/segments_utilisateurs.csv
50000 output/segments_utilisateurs.csv

$ head -3 output/segments_utilisateurs.csv
u000001,Senegal,26,60487,intensif
u000002,Canada,17,26137,occasionnel
u000003,Cote_d_Ivoire,27,45954,actif
```

```
$ cut -d, -f5 output/segments_utilisateurs.csv | sort | uniq -c
  30901 actif
   7430 intensif
  11669 occasionnel
```

Vérification indépendante de Hive. Le comptage des segments a été refait directement sur le fichier exporté, avec des outils Unix, sans passer par Hive. Il redonne exactement la répartition calculée dans Hive (30 901 / 7 430 / 11 669, soit 50 000 au total). Le fichier transmis à MongoDB est donc fidèle à l'agrégation : rien n'a été perdu ni altéré par `getmerge`.

Justification

- `getmerge` concatène les fichiers produits par les *reducers* en un seul CSV, qui est le format attendu par le script MongoDB.
- Le fichier n'a **pas d'en-tête**. Le contrat d'interface (`README_EQUIPE.md`) fixe l'ordre des colonnes : `user_id, country, event_count, total_watch_sec, segment`.
- **Pourquoi 50 000 lignes et non 1 000 000.** MongoDB n'a besoin que du résumé par utilisateur pour enrichir un profil. Transférer les événements bruts dupliquerait le rôle de HDFS et alourdirait la base documentaire. Le volume est divisé par 20 (20 événements par utilisateur en moyenne).
- Ce fichier est le **premier point de reprise** du pipeline (voir D2) : en cas d'échec côté MongoDB, on repart de ce fichier sans relancer Hive.

  Important pour l'étape suivante (B2). Le fichier `segments_utilisateurs.csv` **ne contient pas de ligne d'en-tête**

  La première ligne (`u000001,Senegal,26,60487,intensif`) est déjà un utilisateur. Le script MongoDB doit fournir lui-même les noms des colonnes, dans cet ordre : `user_id, country, event_count, total_watch_sec, segment`, sinon, cette première ligne serait prise pour un en-tête, et seuls 49 999 profils seraient enrichis au lieu de 50 000.

Note de réexécution (25 octobre 2026). La partie A a été réexécutée en entier avec la version finale de `01_hive_analyse.sql` (A1, A2 et EXPLAIN ajoutés). La logique A3 ci-dessus n'a pas changé : le fichier `segments_utilisateurs.csv` obtenu est identique, octet pour octet, à celui consommé par MongoDB (même empreinte SHA-256 `cd53b4ffcda7fb48…`, 50 000 lignes). Sur un poste Windows, `hdfs dfs -getmerge` directement vers le dossier monté `output/` échoue (`Operation not permitted`), parce que le client Hadoop tente de fixer les permissions POSIX du fichier local. On fusionne alors dans `/tmp` du conteneur, puis on copie le fichier, sans changer son contenu (voir `README_EQUIPE.md`).

Partie B – MongoDB

1. Objet et périmètre

La partie B transforme les résultats agrégés de Hive en profils documentaires enrichis, puis sélectionne une audience destinée à Cassandra. Elle comprend B1 (importation et indexation), B2 (enrichissement et étude de performance) et B3 (exportation JSONL). Les observations ci-dessous proviennent des exécutions et captures réalisées sur Windows PowerShell avec Docker Compose et MongoDB.

2. Architecture et flux de données

- Entrées : data/profils_utilisateurs.jsonl.gz (profils documentaires) et workspace/output/segments_utilisateurs.csv (50 000 lignes produites par Hive).
- **Traitement** : script workspace/02_mongodb_enrichissement.py, connecté à la base inf6303_devoir1, collection profils.
- **Sortie** : workspace/output/audience_cible.jsonl, consommable par la partie Cassandra après vérification d'intégration.

Composant	Fonction	Interface
Hive / CSV	Agrégation des événements et segmentation	user_id, country, event_count, total_watch_sec, segment
MongoDB	Stockage des profils, index, enrichissement et filtrage	Collection profils / sous-document analytics
Cassandra	Publication opérationnelle (partie C)	audience_cible.jsonl

3. B1 — Importation documentaire et indexation

Le script a importé 50 000 profils depuis le fichier JSONL compressé. La vérification dans MongoDB a confirmé 50 000 documents. Les champs documentaires imbriqués, notamment preferences et devices, ont été conservés dans les exemples contrôlés. Deux index ont été créés : un index unique sur user_id et un index composé sur consent_analytics puis analytics.segment. L'index unique contribue à empêcher les doublons d'identifiant ; le second soutient le filtre de ciblage.



Figure 1 — Exemple de profil MongoDB avec champs imbriqués (capture d'exécution).

4. B2 — Enrichissement des profils

Le fichier Hive a été lu sans en-tête et rapproché des documents MongoDB par user_id. Les indicateurs ont été placés dans analytics : nb_events, total_watch_sec et segment. Le journal d'exécution indique 50 000 profils MongoDB, 50 000 segments Hive, 50 000 correspondances, 50 000 profils enrichis et aucun segment sans correspondance. L'exemple de profil enrichi confirme la présence des attributs initiaux et du nouveau sous-document analytics.

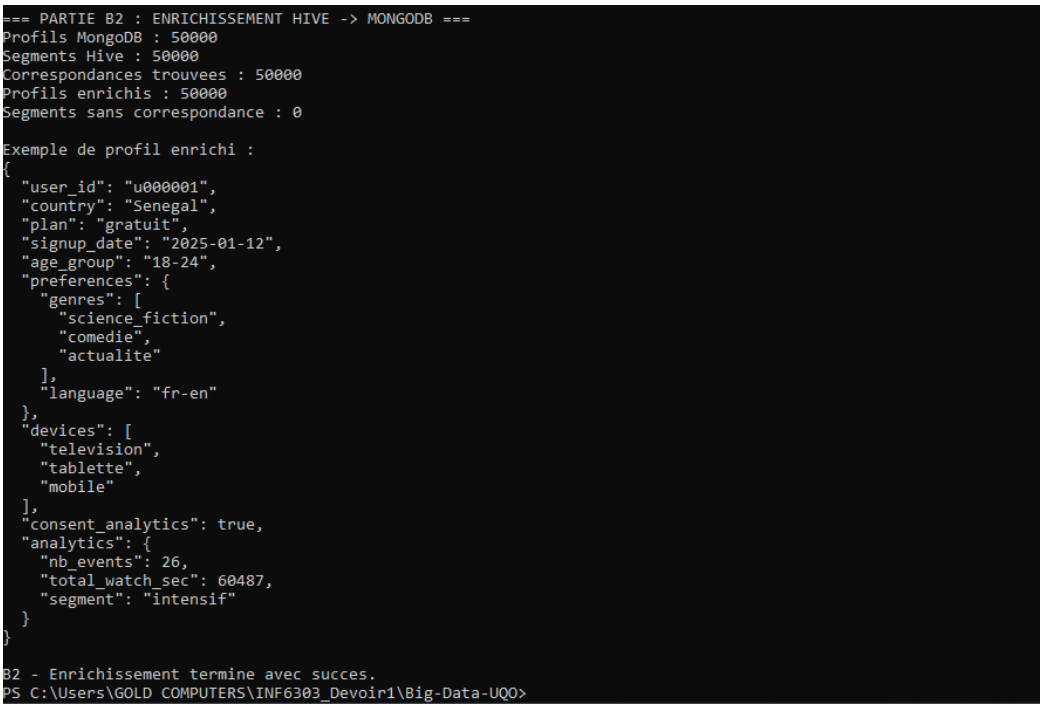


Figure 2 — Profil enrichi : conservation des attributs et ajout du sous-document analytics.

5. Analyse des performances : index composé et parcours naturel

Une requête sur consent_analytics=true et analytics.segment dans {intensif, actif} a été examinée avec explain("executionStats"). Deux exécutions forcées ont été comparées : utilisation de l'index composé et parcours naturel de la collection. Les deux retournent 34 855 documents.

Mesure	Avec index	Parcours naturel
Documents retournés	34 855	34 855
Temps mesuré	69 ms	43 ms
Clés d'index examinées	34 856	0
Documents examinés	34 855	50 000

Interprétation : l'index réduit le nombre de documents examinés, mais ne diminue pas le temps observé sur cet essai. La requête sélectionne près de 70 % de la collection ; le coût de parcours de l'index peut alors dépasser le bénéfice attendu. Ces mesures ponctuelles ne démontrent pas une supériorité générale du parcours naturel : il faudrait répéter les tests, contrôler le cache et comparer plusieurs sélectivités.

```
test> use inf6303_devoir1
switched to db inf6303_devoir1
inf6303_devoir1> const resultat = db.profils.find({
|      consent_analytics: true,
|      "analytics.segment": {
|          $in: ["intensif", "actif"]
|      }
| }).hint("consent_analytics_1_analytics.segment_1").explain("executionStats");

inf6303_devoir1> printjson({
|      nombre_documents_retournes: resultat.executionStats.nReturned,
|      temps_execution_ms: resultat.executionStats.executionTimeMillis,
|      cles_index_examinees: resultat.executionStats.totalKeysExamined,
|      documents_examines: resultat.executionStats.totalDocsExamined
| });
{
  nombre_documents_retournes: 34855,
  temps_execution_ms: 66,
  cles_index_examinees: 34856,
  documents_examines: 34855
}

inf6303_devoir1>
```

Figure 3 — Statistiques explain() de la requête utilisant l'index.

```
inf6303_devoir1> const filtre = {
|      consent_analytics: true,
|      "analytics.segment": {
|          $in: ["intensif", "actif"]
|      }
| };

inf6303_devoir1> const avecIndex = db.profils.find(filtre)
|      .hint("consent_analytics_1_analytics.segment_1")
|      .explain("executionStats");

inf6303_devoir1> const sansIndex = db.profils.find(filtre)
|      .hint({$natural: 1})
|      .explain("executionStats");

inf6303_devoir1> printjson({
|      avec_index: {
|          documents_retournes: avecIndex.executionStats.nReturned,
|          temps_ms: avecIndex.executionStats.executionTimeMillis,
|          cles_examinees: avecIndex.executionStats.totalKeysExamined,
|          documents_examines: avecIndex.executionStats.totalDocsExamined
|      },
|      sans_index: {
|          documents_retournes: sansIndex.executionStats.nReturned,
|          temps_ms: sansIndex.executionStats.executionTimeMillis,
|          cles_examinees: sansIndex.executionStats.totalKeysExamined,
|          documents_examines: sansIndex.executionStats.totalDocsExamined
|      }
| });
{
  avec_index: {
    documents_retournes: 34855,
    temps_ms: 69,
    cles_examinees: 34856,
    documents_examines: 34855
  },
  sans_index: {
    documents_retournes: 34855,
    temps_ms: 43,
    cles_examinees: 0,
    documents_examines: 50000
  }
}

inf6303_devoir1>
```

Figure 4 — Comparaison expérimentale avec et sans index.

6. B3 — Sélection et exportation de l'audience

Le filtre impose simultanément le consentement analytique et un segment actif ou intensif. Le document exporté conserve user_id, country, plan et analytics ; le champ _id et les attributs documentaires non requis ne sont pas transférés. L'export est au format JSON Lines (un objet JSON par ligne), en UTF-8.

Indicateur	Valeur
Profils initiaux	50 000
Profils admissibles / exportés	34 855
Intensifs	6 784
Actifs	28 071

Indicateur	Valeur
Profils exclus	15 145
Réduction du nombre de lignes	30,29 %
Taille du fichier	4 990 244 octets (~4,76 Mio)

Cette réduction de 30,29 % porte sur le nombre de profils, non sur la taille en octets. Le fichier constitue le contrat d'interface vers Cassandra ; son ingestion effective par la partie C n'a pas été démontrée pendant cette phase.

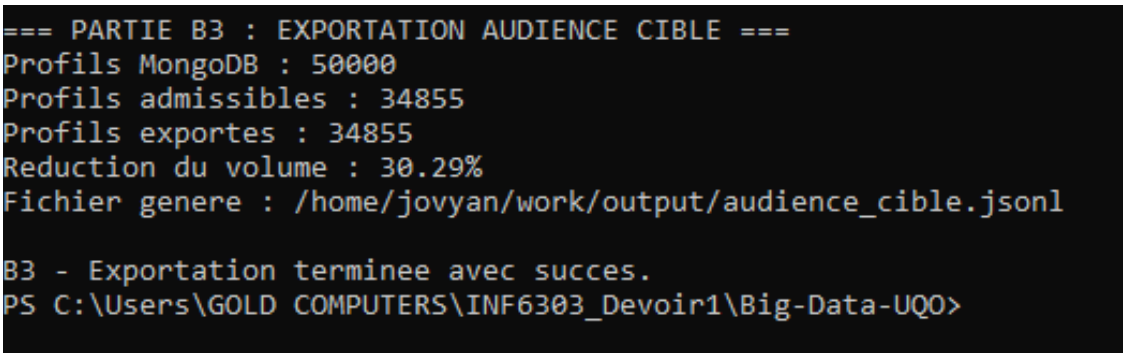


Figure 5 — Journal d'exécution B3 : 34 855 profils exportés.

7. Contrôles de qualité et reproductibilité

Le script workspace/validation_mongodb.py a parcouru les 34 855 lignes, contrôlé la syntaxe JSON, les champs requis, les identifiants uniques, les types des indicateurs et les segments autorisés. Résultat : 34 855 identifiants uniques, 6 784 profils intensifs, 28 071 actifs et zéro erreur. Le consentement a été contrôlé par le filtre MongoDB avant export ; il n'est pas directement revérifiable dans le JSONL, car ce champ n'y figure pas.

Commandes de reproduction (depuis le répertoire du dépôt, services notebook et mongo démarrés) :

```
docker compose up -d --build notebook mongo
docker compose exec notebook python /home/jovyan/work/02_mongodb_enrichissement.py
docker compose exec notebook python /home/jovyan/work/validation_mongodb.py
```



Figure 6 — Contrôle des lignes et de la taille du fichier JSONL.

8. Difficultés rencontrées et impacts

Difficulté	Résolution / conséquence
Construction Docker : cassandra-driver==3.29.2 échoue avec Python 3.13	Contrainte remplacée par cassandra-driver>=3.30,<4 ; reconstruction réussie. Déblocage de l'environnement notebook.
Téléchargement et extraction d'images volumineuses	Démarrage initial prolongé ; les couches Docker en cache ont accéléré la reconstruction suivante.

Difficulté	Résolution / conséquence
Interopérabilité CSV → documents MongoDB	Correspondance par user_id et insertion des résultats dans analytics ; 50 000 correspondances.
Index moins rapide lors du test	69 ms contre 43 ms ; nécessité d'interpréter les coûts de requête et la forte proportion de résultats.
Avertissements Git : trailing whitespace et LF/CRLF	Nettoyage des espaces terminaux, contrôle `git diff --cached --check` et validation syntaxique Python ; aucun blocage à la publication.

L'erreur de construction Docker a été le blocage principal. Les avertissements de fins de ligne Git n'étaient pas des erreurs fonctionnelles. La correction du Dockerfile est partagée et devra être revue par l'équipe pour préserver la compatibilité des autres étapes.

9. Limites, intégration et conclusion

La validation a porté sur la sortie MongoDB et non sur l'exécution complète du pipeline à partir d'un environnement vierge. Les mesures explain() n'ont pas été répétées sur plusieurs séries ; elles restent indicatives. Le calcul du score utilisé par Cassandra n'est pas défini dans la partie B et doit être convenu avec la personne responsable de la partie C.

La contribution MongoDB est implémentée, vérifiée et publiée dans une branche dédiée du dépôt collectif (commit e7555e8, Pull Request #2 ouverte au moment de la capture). La fusion et l'intégration finale avec Cassandra restent du ressort de l'équipe. Les résultats attestent la transformation de 50 000 profils en une audience ciblée de 34 855 documents JSONL, sans erreur détectée dans les contrôles exécutés.

Note d'intégration (25 octobre 2026). Depuis la rédaction de cette partie :

- La Pull Request #2 a été fusionnée dans `main`.
- Le score utilisé par Cassandra est fixé à `analytics.total_watch_sec`, la durée totale calculée par Hive (voir la partie C).
- L'ingestion de `audience_cible.jsonl` par Cassandra est démontrée en C2 (34 855 documents exportés = 34 855 lignes chargées), et la cohérence des valeurs est vérifiée de bout en bout en D1.
- Ordre des champs de l'index composé : le champ testé par égalité (`consent_analytics`) est placé en premier, puis le champ testé par `$in` (`analytics.segment`). MongoDB parcourt ainsi deux intervalles contigus de l'index, `(true, actif)` et `(true, intensif)`, au lieu de filtrer toute la collection. C'est ce que confirment les 34 856 clés examinées pour 34 855 documents retournés.

Partie C – Cassandra

Script : `workspace/03_cassandra_publication.py` · Sortie complète : `workspace/output/log_C.txt`

```
docker compose --profile nosql up -d --build
# attendre que le service cassandra soit "healthy" (~2 min)
docker compose exec notebook python /home/jovyan/work/03_cassandra_publication.py
```

Entrée : `output/audience_cible.jsonl` produit par MongoDB (partie B3), 34 855 documents. Le script est rejouable : il supprime et recrée ses tables à chaque exécution.

C1. Modèle orienté requête

Requête à servir

« Top N des utilisateurs d'un pays et d'un segment, triés par score décroissant. »

Table

```
CREATE KEYSPACE IF NOT EXISTS inf6303_devoir1
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE audience_par_pays_segment (
    country   text,
    segment   text,
    score     int,      -- = analytics.total_watch_sec
    user_id   text,
    nb_events int,      -- = analytics.nb_events
    plan      text,
    PRIMARY KEY ((country, segment), score, user_id)
) WITH CLUSTERING ORDER BY (score DESC, user_id ASC);
```

Choix du score

`score = analytics.total_watch_sec`, la durée totale de visionnement calculée par Hive (`SUM(watch_seconds)`) et transmise par MongoDB. C'est la métrique qui définit le segment (intensif ≥ 55 000 s, actif ≥ 30 000 s). Elle est traçable de bout en bout (Hive → MongoDB → Cassandra) et aucune valeur n'est inventée entre les étapes.

Clé de partition `(country, segment)`

- La requête fournit toujours un pays **et** un segment : ce couple est donc la clé de partition.
- Toutes les lignes d'un couple sont stockées ensemble, sur le même nœud (et ses réplicas). Une requête de service lit **une seule partition**, sans balayer le cluster.
- Une clé `country` seule obligerait à filtrer le segment. Une clé `segment` seule ne donnerait que 2 partitions énormes.

Clustering `(score DESC, user_id ASC)`

- Dans une partition, Cassandra garde les lignes triées sur disque selon les colonnes de clustering. Avec `score DESC`, le tri demandé est déjà fait à l'écriture : la lecture se limite aux N premières lignes (`LIMIT 10`), sans aucun tri ni `ORDER BY` à l'exécution.
- **Égalités de score réglées par `user_id`.** En Cassandra, un `INSERT` est un *upsert* : deux lignes qui ont la même clé primaire s'écrasent. Or plusieurs utilisateurs d'une même partition ont exactement le même `total_watch_sec`. Mesuré sur `audience_cible.jsonl` :

- 2 548 combinaisons `(country, segment, score)` sont partagées par au moins deux utilisateurs ;
- jusqu'à 5 ex aequo, par exemple Canada/actif/38 158 s : u001872, u003108, u004232, u005460, u027798 ;
- avec la clé `((country, segment), score)`, **2 772 lignes auraient été perdues en silence** (32 083 lignes chargées au lieu de 34 855) et le contrôle C2 aurait échoué.

Ajouter `user_id`, déjà unique, comme dernière colonne de clustering rend la clé primaire unique. Le tri secondaire par `user_id ASC` rend aussi l'ordre des ex aequo déterministe.

Dénormalisation

- Cassandra ne fait pas de jointure. La table contient donc **tout ce que la réponse affiche** : `user_id`, `country`, `segment`, `score`, `nb_events`, `plan`. Une lecture suffit, sans aller chercher le plan dans MongoDB ni la durée dans Hive.
- La table est une **vue de service** dérivée de MongoDB : `plan` y est copié, et `country` et `segment` font partie de la clé. Si un besoin de lecture change, on crée une autre table avec les mêmes données et une autre clé (voir `audience_par_plan` en C3). Le stockage dupliqué est le prix d'une latence prévisible.
- Seul le sous-ensemble utile est transféré : 34 855 profils consentants intensif/actif. Le million d'événements reste dans HDFS.
- Contrepartie : les copies doivent être tenues cohérentes. Ici, elles sont reconstruites à chaque exécution à partir de `audience_cible.jsonl`, qui est le point de reprise.

Risque de partition chaude

Les 16 partitions (8 pays × 2 segments) sont déséquilibrées (comptages tirés de `log_C.txt`) :

Pays	actif	intensif
Belgique	1 978	474
Cameroun	2 536	604
Canada	**8 792**	2 077
Cote_d_Ivoire	3 370	811
France	3 305	820
Maroc	2 752	684
Senegal	3 666	932
Tunisie	1 672	**382**
Total	28 071	6 784

- Canada/actif = **8 792** lignes contre **382** pour Tunisie/intensif, soit environ **23 fois plus**. Le Canada représente à lui seul 31 % des lignes (10 869 / 34 855), comme dans les données sources (311 247 événements sur 1 000 000).
- Le nœud qui détient Canada/actif reçoit une part disproportionnée des lectures et des écritures (partition chaude). Avec seulement 16 partitions, la répartition par hachage est aussi grossière : quelques nœuds porteraient tout.
- **À ce volume, le risque reste théorique.** 8 792 lignes de quelques dizaines d'octets font moins de 1 Mo, très loin de la limite pratique d'environ 100 Mo par partition. De plus, la requête ne lit que les 10 premières lignes, déjà triées.
- **Si le volume augmentait**, on ajouterait un *bucket* à la clé de partition, `((country, segment, bucket), score, user_id)` avec `bucket = hash(user_id) % N`. Le top 10 global s'obtient alors en lisant les N buckets et en fusionnant N petites listes déjà triées. Autre option : remplacer le segment par une granularité plus fine (une tranche de score).

C2. Chargement

- Lecture ligne à ligne de `/home/jovyan/work/output/audience_cible.jsonl`. Correspondance des champs : `country`, `analytics.segment`, `analytics.total_watch_sec → score`, `user_id`, `analytics.nb_events → nb_events`, `plan`.
- **Requête préparée** (`session.prepare("INSERT ... VALUES (?, ?, ?, ?, ?, ?)")`). Le serveur analyse le CQL une seule fois, puis seules les valeurs circulent pour chaque ligne. C'est plus rapide et il n'y a aucune concaténation de chaînes, donc aucun risque d'injection CQL. Les insertions sont envoyées en parallèle (`execute_concurrent_with_args`, concurrence 64) : 34 855 insertions en 13,8 s, 0 échec.
- **Comptage par partition** : un `SELECT COUNT(*) ... WHERE country = ? AND segment = ?` pour chacun des 16 couples, c'est-à-dire la même méthode d'accès que le service. On évite ainsi un `COUNT(*)` global qui balaierait tout le cluster.

Contrôle	Valeur
Documents exportés par MongoDB (`audience_cible.jsonl`)	**34 855**
Lignes chargées dans `audience_par_pays_segment`	**34 855**
Partitions	16
Écart	0 (`assert nb_charges == nb_exportes` réussi)

Chaque partition contient exactement le nombre de documents attendu pour son couple.

C3. Requêtes de service

Requête préparée commune (une seule partition lue, ordre déjà sur disque) :

```
SELECT user_id, score, nb_events, plan
FROM audience_par_pays_segment
WHERE country = ? AND segment = ?
LIMIT 10;
```

Top 10 Canada / intensif

#	user_id	score (s)	nb_events	plan
1	u012192	99 744	36	standard
2	u047861	96 938	28	standard
3	u025691	95 497	34	standard
4	u049062	95 407	42	premium
5	u042077	94 314	30	standard
6	u001658	94 019	24	standard
7	u007230	93 381	26	gratuit
8	u013563	93 250	37	gratuit
9	u025640	90 301	31	gratuit
10	u029858	90 175	33	premium

Vérification indépendante : un tri Python du fichier `audience_cible.jsonl` (Canada, intensif, score décroissant) donne exactement la même liste.

Autre couple pertinent : France / actif

Choix : la France est le premier marché européen du jeu de données (118 790 événements contre 69 962 pour la Belgique), et le segment actif est le plus nombreux. Les premiers actifs sont juste sous le seuil intensif (55 000 s) : c'est l'audience la plus facile à faire passer à l'intensif par une campagne ciblée.

#	user_id	score (s)	nb_events	plan
1	u048934	54 994	25	premium
2	u009679	54 993	22	gratuit
3	u015351	54 982	24	standard
4	u017027	54 967	21	standard
5	u020613	54 964	24	gratuit
6	u010041	54 951	22	standard
7	u045319	54 948	22	gratuit
8	u001195	54 941	27	gratuit
9	u044077	54 938	23	gratuit
10	u036149	54 929	26	standard

Tous les scores sont < 55 000, ce qui confirme que la bonne frontière de segment est respectée dans Cassandra.

Recherche globale par plan : pas naturelle pour cette table

```
SELECT user_id, country, segment, score
FROM audience_par_pays_segment WHERE plan = 'premium' LIMIT 10;
```

Réponse de Cassandra :

```
code=2200 [Invalid query] message="Cannot execute this query as it might involve data
filtering and thus may have unpredictable performance. If you want to execute this query
despite the performance unpredictability, use ALLOW FILTERING"
```

`plan` ne fait ni partie de la clé de partition ni du clustering. Cassandra ne sait donc pas quelles partitions lire : il faudrait les ouvrir toutes. Il refuse plutôt que de lancer en silence un balayage complet. Avec `ALLOW FILTERING`, la requête passe et trouve **6 352** lignes premium, mais seulement après avoir lu les **34 855** lignes des 16 partitions. Le coût est proportionnel à la taille de la table, pas à celle de la réponse, et le résultat n'est même pas trié globalement par score (chaque partition est triée séparément). C'est l'inverse du principe « une requête = une partition ».

Table supplémentaire proposée : `audience_par_plan`

```
CREATE TABLE audience_par_plan (
    plan      text,
    score     int,
    user_id   text,
    country   text,
    segment   text,
    nb_events int,
    PRIMARY KEY ((plan), score, user_id)
) WITH CLUSTERING ORDER BY (score DESC, user_id ASC);
```

- Clé de partition `plan` : la nouvelle requête fournit le plan, donc une requête = une partition.
- Même clustering `(score DESC, user_id ASC)` : top N par score sans tri, et ex aequo conservés.
- Mêmes données dupliquées depuis `audience_cible.jsonl` (dénormalisation : une table par requête).

Résultats (chargement par requête préparée, comptage par partition) :

plan	lignes chargées	attendu (fichier)
gratuit	16 831	16 831
premium	6 352	6 352
standard	11 672	11 672
Total	**34 855**	**34 855**

Le nombre de premium (6 352) est identique à celui obtenu par `ALLOW FILTERING` : les deux tables sont cohérentes.

Top 10 premium (`WHERE plan = 'premium' LIMIT 10`, une seule partition) :

#	user_id	score (s)	country	segment
1	u041488	103 073	Cameroun	intensif
2	u040743	96 078	Cote_d_Ivoire	intensif
3	u049062	95 407	Canada	intensif
4	u009818	95 404	Senegal	intensif
5	u049503	92 788	France	intensif
6	u025368	92 400	Cameroun	intensif
7	u024406	92 099	Belgique	intensif
8	u002854	90 899	Senegal	intensif
9	u029858	90 175	Canada	intensif
10	u004019	89 320	Canada	intensif

u049062 et u029858 apparaissent dans les deux tops (Canada/intensif et premium) avec le même score : c'est la même donnée vue par deux clés différentes.

Limite de cette clé : elle ne donne que 3 partitions, dont `gratuit` qui regroupe 48 % des lignes. C'est acceptable à 34 855 lignes. À plus grande échelle, on utiliserait `((plan, country), score, user_id)` si le pays est connu à la requête, ou un *bucket* `((plan, bucket), score, user_id)` pour garder une recherche globale par plan.

Partie D – Validation et architecture

D1. Traçabilité

Script : `workspace/04_validation_pipeline.py` · Sortie complète : `workspace/output/log_D.txt`

```
docker compose --profile nosql up -d --build
# attendre que le service cassandra soit "healthy" (~2 min)
docker compose exec notebook python /home/jovyan/work/04_validation_pipeline.py
```

Prérequis : les parties A, B et C ont été exécutées (CSV Hive présent, collection MongoDB `profils` enrichie, table Cassandra `audience_par_pays_segment` chargée). Le script ne modifie rien : il lit les quatre étapes et échoue (code de sortie non nul) au moindre écart. D'autres `user_id` peuvent être suivis en argument : `python 04_validation_pipeline.py u000004 u000006`.

D1.1 Comptages à chaque étape

Chaque étape est relue dans son propre système, sans réutiliser les compteurs des scripts précédents :

- **Hive** : lecture de `output/segments_utilisateurs.csv` (résultat du `getmerge`).
- **MongoDB** : `count_documents` sur la collection `inf6303_devoir1.profils`.
- **Fichier de passage** : lecture de `output/audience_cible.jsonl`.
- **Cassandra** : lecture des partitions par leur clé `(country, segment)`, comme le fait le service. On lit les 8 pays × 3 segments, soit 24 partitions, et non un `COUNT(*)` global qui balaierait tout le cluster. Les 8 partitions `occasionnel` sont lues pour prouver qu'elles sont vides.

Étape	Système	Lignes
1	Hive : `segments_utilisateurs.csv`	**50 000**
2	MongoDB : profils	**50 000**
	MongoDB : profils enrichis (`analytics` présent)	**50 000**
	MongoDB : consentants intensif/actif (filtre B3)	34 855
3	`audience_cible.jsonl`	**34 855**
4	Cassandra : `audience_par_pays_segment` (16 partitions non vides)	**34 855**

Pourquoi on passe de 50 000 à 34 855

Le filtre B3 (`consent_analytics = true` et `analytics.segment ∈ {intensif, actif}`) est la seule réduction du pipeline. Les 15 145 profils exclus se décomposent ainsi :

Segment Hive	Consentants	Non consentants
intensif	6 784 → exportés	646 → exclus
actif	28 071 → exportés	2 830 → exclus
occasionnel	10 642 → exclus	1 027 → exclus

- Exclus car occasionnel : 10 642 + 1 027 = **11 669** (le même nombre que la sortie Hive A3).
- Exclus car intensif ou actif mais non consentants : 646 + 2 830 = **3 476**.

- 34 855 + 11 669 + 3 476 = **50 000** : aucun profil ne disparaît sans raison.

Contrôles de cohérence (tous réussis)

Les contrôles comparent les **valeurs**, pas seulement les nombres de lignes, et portent sur **tous** les utilisateurs, pas seulement sur les trois suivis.

Contrôle	Obtenu	Attendu
Hive : `user_id` distincts	50 000	50 000
Hive : somme des `nb_events`	1 000 000	1 000 000
MongoDB : profils = lignes Hive	50 000	50 000
MongoDB : profils enrichis = profils	50 000	50 000
JSONL : lignes = admissibles MongoDB	34 855	34 855
JSONL + exclus = profils	50 000	50 000
Cassandra : lignes = lignes JSONL	34 855	34 855
Cassandra : lignes `occasionnel`	0	0
Hive → MongoDB : profils dont pays ou `analytics` diffère	0	0
MongoDB → JSONL : documents dont pays, plan ou `analytics` diffère	0	0
JSONL → Cassandra : lignes dont une colonne diffère	0	0
Cassandra : `user_id` absents du JSONL	0	0

La somme des `nb_events` vaut 1 000 000 : chaque événement Hive est compté une fois, et seul cet agrégat quitte HDFS. Le million d'événements n'est chargé ni dans MongoDB ni dans Cassandra.

D1.2 Traçabilité de trois `user_id`

Les trois utilisateurs sont réels et pris dans les fichiers produits, sans aucun identifiant fabriqué. Ils couvrent les trois cas du pipeline :

`user_id`	Cas
u000001	intensif, consentant → publié
u000003	actif, consentant → publié
u000002	occasionnel **et** non consentant → arrêté dans MongoDB

Tableau comparatif

`duree/score` = `total_watch_sec` dans Hive, MongoDB et le JSONL ; `score` dans Cassandra.

`user_id`	Étape	pays	nb_events	durée / score (s)	segment	plan	consentement
u000001	Hive (CSV)	Senegal	26	60 487	intensif	–	–

user_id	Étape	pays	nb_events	durée / score (s)	segment	plan	consentement
	MongoDB	Senegal	26	60 487	intensif	gratuit	true
	audience_cible.jsonl	Senegal	26	60 487	intensif	gratuit	–
	Cassandra	Senegal	26	60 487	intensif	gratuit	–
u000003	Hive (CSV)	Cote_d_Ivoire	27	45 954	actif	–	–
	MongoDB	Cote_d_Ivoire	27	45 954	actif	standard	true
	audience_cible.jsonl	Cote_d_Ivoire	27	45 954	actif	standard	–
	Cassandra	Cote_d_Ivoire	27	45 954	actif	standard	–
u000002	Hive (CSV)	Canada	17	26 137	occasionnel	–	–
	MongoDB	Canada	17	26 137	occasionnel	premium	**false**
	audience_cible.jsonl	*absent*					
	Cassandra	*absent*					

Explication des différences

u000001 (Senegal, intensif) : les valeurs sont identiques aux quatre étapes.

- 60 487 s ≥ 55 000 s, donc segment intensif, conformément au seuil Hive.
- Dans Cassandra, la ligne se trouve dans la partition `(Senegal, intensif)` avec `score = 60 487`, c'est-à-dire la durée calculée par Hive, recopiée sans transformation.

u000003 (Cote_d_Ivoire, actif) : les valeurs sont identiques aux quatre étapes.

- 30 000 s ≤ 45 954 s < 55 000 s, donc segment actif.
- La ligne se trouve dans la partition `(Cote_d_Ivoire, actif)`.

u000002 (Canada, occasionnel) : présent dans Hive et MongoDB, absent ensuite. C'est le comportement attendu, pas une perte de données.

- Il est **enrichi** dans MongoDB : le sous-document `analytics` reprend exactement les valeurs Hive. MongoDB garde le profil complet de tous les utilisateurs (document opérationnel).
- Il est **exclu par le filtre B3** pour deux raisons indépendantes : segment occasionnel (26 137 s < 30 000 s) et `consent_analytics = false`. Une seule aurait suffi.
- Il n'arrive donc ni dans `audience_cible.jsonl` ni dans Cassandra. Le script l'a cherché dans les 24 partitions de la table, y compris `(Canada, occasionnel)`, qui est vide.
- Cas complémentaires, vérifiés avec `04_validation_pipeline.py u000004 u000006` :
  - u000004 (Canada, actif, 51 017 s) est exclu **uniquement** pour son consentement (`false`).
  - u000006 (France, occasionnel, 22 735 s) est exclu **uniquement** pour son segment, bien qu'il soit consentant.

Ce qui change d'une étape à l'autre (sans être une incohérence)

Différence	Où	Raison
`plan` et `consent_analytics` apparaissent	MongoDB	Ils ne viennent pas des événements mais du profil (`profils_utilisateurs.jsonl.gz`). Hive ne les connaît pas.
`consent_analytics` disparaît	JSONL, Cassandra	Il a servi de filtre : toutes les lignes exportées valent `true`, le transférer n'apporterait rien.
`total_watch_sec` devient `score`	Cassandra	Il s'agit d'un renommage : c'est la colonne de clustering qui trie le top N (voir partie C). La valeur est identique.
`country` et `segment` deviennent la clé de partition	Cassandra	Ils sont imposés par la requête de service « top d'un pays et d'un segment ».
Le pays existe deux fois en amont	Hive (événements) et MongoDB (profil)	Les deux sources concordent pour les 50 000 utilisateurs (0 écart). Le JSONL reprend le pays du profil.

Conclusion D1

- `user_id` relie bien les trois systèmes : chaque ligne Cassandra se retrouve, avec les mêmes valeurs, dans le JSONL, dans MongoDB et dans la sortie Hive.
- La seule réduction (50 000 → 34 855) est le filtre métier B3, et elle est entièrement expliquée.
- Aucune valeur n'est recalculée ni inventée après Hive : la durée et le nombre d'événements de Cassandra sont ceux de l'agrégation HiveQL.

D2. Architecture

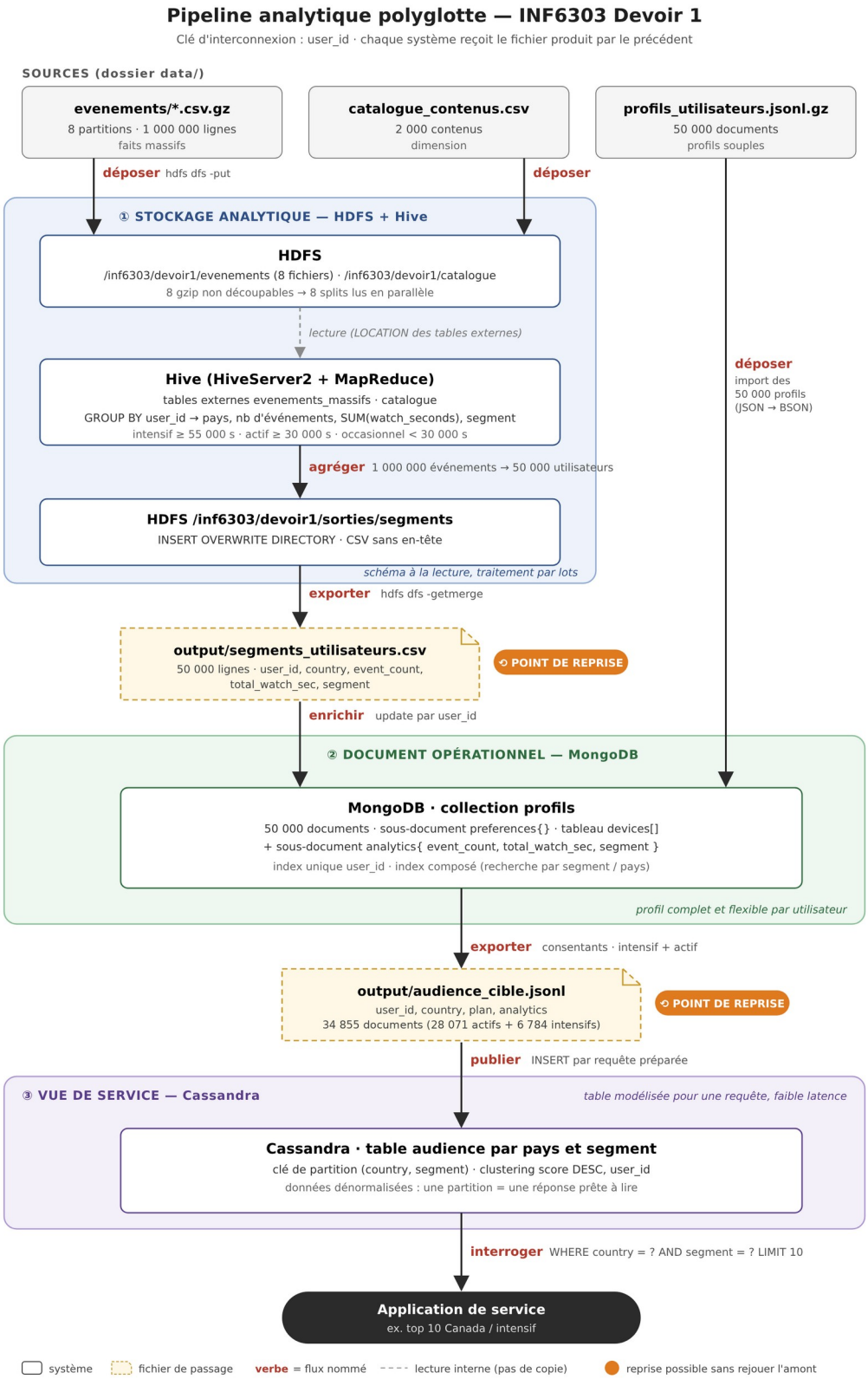


Figure D2. Architecture du pipeline Hive → MongoDB → Cassandra. Les rectangles pleins sont des systèmes, les rectangles pointillés des fichiers de passage, et les verbes en rouge nomment chaque flux.

Flux nommés

#	Flux	De → Vers	Mécanisme	Volume
1	**déposer**	`data/evenements`, `data/catalogue` → HDFS	`00_deposer_dans_hdfs.sh` (`hdfs dfs -put`)	8 fichiers gzip, 1 000 000 lignes ; 2 000 contenus
2	**agréger**	HDFS → Hive → HDFS `/sorties/segments`	tables externes + `GROUP BY user_id` ; `INSERT OVERWRITE DIRECTORY`	1 000 000 événements → 50 000 lignes
3	**exporter**	HDFS → `segments_utilisateurs.csv`	`hdfs dfs -getmerge`	50 000 lignes
—	**déposer** (import)	`profils_utilisateurs.jsonl.gz` → MongoDB	import JSON → BSON	50 000 documents
4	**enrichir**	`segments_utilisateurs.csv` → MongoDB	mise à jour par `user_id` (sous-document `analytics`)	50 000 profils
5	**exporter**	MongoDB → `audience_cible.jsonl`	filtre : consentement + segments intensif/actif	34 855 documents
6	**publier**	`audience_cible.jsonl` → Cassandra	`INSERT` par requête préparée	= nombre de documents exportés
7	**interroger**	Cassandra → application	`WHERE country = ? AND segment = ?`	top N par partition

La clé `user_id` traverse les trois systèmes sans jamais être recréée. Chaque système consomme le fichier produit par le précédent, et aucun identifiant n'est fabriqué à la main entre deux étapes.

Trois rôles distincts

Stockage analytique (HDFS + Hive). HDFS conserve les faits bruts : le million d'événements, en 8 fichiers compressés. Hive leur applique un schéma *à la lecture* au moyen de tables externes, puis les agrège par lots avec MapReduce. Ce couple est fait pour parcourir tout l'historique, pas pour répondre vite à une question précise sur un utilisateur. Les 8 fichiers gzip ne peuvent pas être découpés, mais chacun forme un *split* indépendant, ce qui permet 8 tâches de lecture en parallèle. Le résultat est un **agrégat** : 50 000 lignes au lieu d'un million.

Document opérationnel (MongoDB). MongoDB garde **le profil complet de chaque utilisateur** dans un seul document, avec une structure souple : sous-document `preferences`, tableau `devices`. L'enrichissement ajoute à ce document l'agrégat calculé par Hive, sous forme de sous-document `analytics`. On peut alors lire ou modifier un utilisateur entier en une opération. L'index unique sur `user_id` garantit qu'il y a une seule fiche par utilisateur et rend la mise à jour efficace.

Vue de service (Cassandra). Cassandra ne garde que **ce qu'il faut pour répondre à une requête connue** : « les meilleurs utilisateurs d'un pays et d'un segment ». La table est construite à partir de cette requête. La clé de partition `(country, segment)` place toutes les réponses d'un couple au même endroit, et le *clustering* les range déjà triées par score décroissant. Les données sont dénormalisées : on accepte de dupliquer `country` et `plan` pour obtenir une lecture en une seule partition, à faible latence.

Pourquoi les volumes diminuent à chaque étape

Étape	Données transférées	Justification
Hive → MongoDB	50 000 agrégats, pas 1 000 000 événements	MongoDB n'a besoin que du résumé par utilisateur pour enrichir un profil. Charger les événements bruts dupliquerait le rôle de HDFS.

Étape	Données transférées	Justification
MongoDB → Cassandra	34 855 profils (consentants, intensif + actif)	La vue de service ne concerne que l'audience ciblable. Les utilisateurs occasionnels et ceux qui n'ont pas donné leur consentement sont exclus.

Points de reprise

Le pipeline compte **deux points de reprise**. Ce sont les fichiers de passage écrits sur disque, dans `workspace/output/`, entre deux systèmes.

- `segments_utilisateurs.csv` : si l'enrichissement MongoDB échoue (service arrêté, script interrompu), on relance seulement `02_mongodb_enrichissement.py` sur ce fichier. Il n'est pas nécessaire de refaire l'agrégation Hive sur le million d'événements. La mise à jour par `user_id` est **idempotente** : la rejouer donne le même résultat, sans créer de doublons.
- `audience_cible.jsonl` : si le chargement Cassandra échoue, par exemple parce que Cassandra n'accepte pas encore les connexions , on relance seulement `03_cassandra_publication.py`. Un `INSERT` sur la même clé primaire écrase la ligne existante, donc rejouer le chargement ne crée pas de doublons.

Pour savoir si la reprise est nécessaire, on compare le nombre de lignes du fichier de passage avec le nombre d'éléments chargés dans le système suivant. C'est exactement ce que demandent B2 et C2.

Conclusion

Réponse à la question directrice. Le pipeline évite de confondre les responsabilités, parce que chaque système ne garde que le rôle pour lequel son modèle de données est conçu, et que les systèmes communiquent seulement par des fichiers de passage contrôlés.

- **Hive sur HDFS** porte le volume : le million d'événements y reste. Il est lu en parallèle (8 fichiers gzip), analysé et agrégé en lot, avec un schéma appliqué à la lecture. C'est le seul endroit où le détail des événements existe.
- **MongoDB** porte le document opérationnel : le profil complet et souple de chaque utilisateur (préférences, appareils, consentement), enrichi par les indicateurs de Hive dans un sous-document `analytics`. C'est là que s'appliquent les règles métier, comme le consentement et le ciblage par segment.
- **Cassandra** porte la vue de service : une table dénormalisée conçue pour une requête précise. La clé `((country, segment), score, user_id)` permet de lire une seule partition, déjà triée. Un nouveau besoin de lecture se traite avec une nouvelle table (`audience_par_plan`), pas avec un filtrage.

Les données transférées diminuent à chaque étape : 1 000 000 événements → 50 000 agrégats → 34 855 profils ciblés. Chaque réduction est justifiée par le rôle de l'étape suivante. `user_id` relie les trois systèmes, et la validation D1 montre qu'aucune valeur n'est perdue, recalculée ou inventée entre Hive et Cassandra : tous les contrôles sont réussis, avec 0 écart sur les 50 000 utilisateurs. Enfin, les fichiers de passage servent de points de reprise : en cas d'échec entre deux systèmes, on rejoue seulement l'étape suivante, sans recalculer les précédentes.

Limites. L'environnement est mono-nœud (facteur de réplication 1 dans Cassandra, un seul serveur MongoDB). Le déséquilibre des partitions Cassandra (Canada/actif environ 23 fois plus grand que Tunisie/intensif) et la faible sélectivité de l'index composé MongoDB n'ont pas d'effet à ce volume, mais devraient être revus à plus grande échelle (bucketing des partitions, index sur des requêtes plus sélectives).

Annexes – preuves d'exécution

Annexe 1 – Sortie de `04_validation_pipeline.py` (`output/log_D.txt`)

```
=== FICHIERS DE PASSAGE ===
segments_utilisateurs.csv OK 1654528
audience_cible.jsonl OK 4990244

=== D1.1 : COMPTAGES PAR ETAPE ===
Etape                                          Lignes
1. Hive : segments_utilisateurs.csv             50000
2. MongoDB : profils                            50000
   MongoDB : profils enrichis (analytics)       50000
   MongoDB : consentants intensif/actif         34855
3. Fichier : audience_cible.jsonl               34855
4. Cassandra : audience_par_pays_segment        34855
   Partitions non vides : 16

Reduction par filtre MongoDB :
  intensif     consentants   6784   non consentants    646
  actif        consentants  28071   non consentants   2830
  occasionnel  consentants  10642   non consentants   1027
  Exclus car occasionnel : 11669
  Exclus car intensif/actif non consentant : 3476

Controles de coherence :
  OK     Hive : user_id distincts                              50000  (attendu 50000)
  OK     Hive : somme des nb_events                          1000000  (attendu 1000000)
  OK     MongoDB : profils = lignes Hive                       50000  (attendu 50000)
  OK     MongoDB : profils enrichis = profils                  50000  (attendu 50000)
  OK     JSONL : lignes = admissibles MongoDB                  34855  (attendu 34855)
  OK     JSONL + exclus = profils                              50000  (attendu 50000)
  OK     Cassandra : lignes = lignes JSONL                     34855  (attendu 34855)
  OK     Cassandra : lignes occasionnel                            0  (attendu 0)
  OK     Hive -> MongoDB : profils differents                      0  (attendu 0)
  OK     MongoDB -> JSONL : documents differents                   0  (attendu 0)
  OK     JSONL -> Cassandra : lignes differentes                   0  (attendu 0)
  OK     Cassandra : user_id absents du JSONL                      0  (attendu 0)

=== D1.2 : TRACABILITE DE 3 user_id ===

--- u000001 ---
etape                  | pays           | nb_events | duree/score | segment     | plan
| consent
-----------------------+----------------+-----------+-------------+-------------
+-----------+--------
Hive (CSV)             | Senegal        | 26        | 60487       | intensif    | -
| -
MongoDB (profils)      | Senegal        | 26        | 60487       | intensif    | gratuit
| True
audience_cible.jsonl   | Senegal        | 26        | 60487       | intensif    | gratuit
| -
Cassandra              | Senegal        | 26        | 60487       | intensif    | gratuit
| -
=> Present aux 4 etapes, valeurs identiques. Le score Cassandra est la duree totale Hive
(total_watch_sec) ; le plan vient du profil MongoDB.

--- u000003 ---
etape                  | pays           | nb_events | duree/score | segment     | plan
```

```
| consent
----------------------+----------------+-----------+------------+-------------
+-----------+--------
Hive (CSV)             | Cote_d_Ivoire  | 27        | 45954      | actif       | -
| -
MongoDB (profils)      | Cote_d_Ivoire  | 27        | 45954      | actif       |
standard  | True
audience_cible.jsonl   | Cote_d_Ivoire  | 27        | 45954      | actif       |
standard  | -
Cassandra              | Cote_d_Ivoire  | 27        | 45954      | actif       |
standard  | -
=> Present aux 4 etapes, valeurs identiques. Le score Cassandra est la duree totale Hive
(total_watch_sec) ; le plan vient du profil MongoDB.

--- u000002 ---
etape                  | pays           | nb_events | duree/score | segment     | plan
| consent
----------------------+----------------+-----------+------------+-------------
+-----------+--------
Hive (CSV)             | Canada         | 17        | 26137      | occasionnel | -
| -
MongoDB (profils)      | Canada         | 17        | 26137      | occasionnel | premium
| False
audience_cible.jsonl   | ABSENT         |           |            |             |
|
Cassandra              | ABSENT         |           |            |             |
|
=> Absent du JSONL et de Cassandra, comme attendu : segment occasionnel et
consent_analytics = false. Le filtre B3 l'exclut ; il reste enrichi dans MongoDB.

VALIDATION DU PIPELINE REUSSIE : Hive -> MongoDB -> Cassandra coherent
```

Annexe 2 – Sortie de `03_cassandra_publication.py` (`output/log_C.txt`)

```
=== PARTIE C1 : MODELE ORIENTE REQUETE ===
Connexion Cassandra etablie
Table audience_par_pays_segment creee :
  PRIMARY KEY ((country, segment), score, user_id)
  CLUSTERING ORDER BY (score DESC, user_id ASC)

=== PARTIE C2 : CHARGEMENT ===
Documents lus dans audience_cible.jsonl : 34855
Insertions envoyees : 34855, echecs : 0 (13.8 s)

Lignes par partition (country, segment) :
  Belgique       actif       1978  (attendu 1978)
  Belgique       intensif     474  (attendu 474)
  Cameroun       actif       2536  (attendu 2536)
  Cameroun       intensif     604  (attendu 604)
  Canada         actif       8792  (attendu 8792)
  Canada         intensif    2077  (attendu 2077)
  Cote_d_Ivoire  actif       3370  (attendu 3370)
  Cote_d_Ivoire  intensif     811  (attendu 811)
  France         actif       3305  (attendu 3305)
  France         intensif     820  (attendu 820)
  Maroc          actif       2752  (attendu 2752)
  Maroc          intensif     684  (attendu 684)
  Senegal        actif       3666  (attendu 3666)
  Senegal        intensif     932  (attendu 932)
  Tunisie        actif       1672  (attendu 1672)
  Tunisie        intensif     382  (attendu 382)
Nombre de partitions : 16
```

```
Documents exportes par MongoDB : 34855
Lignes chargees dans Cassandra : 34855
C2 - Chargement verifie : exportes = charges

=== PARTIE C3 : REQUETES DE SERVICE ===

--- Top 10 Canada / intensif ---
        user_id |           score |       nb_events |            plan
        u012192 |           99744 |              36 |        standard
        u047861 |           96938 |              28 |        standard
        u025691 |           95497 |              34 |        standard
        u049062 |           95407 |              42 |         premium
        u042077 |           94314 |              30 |        standard
        u001658 |           94019 |              24 |        standard
        u007230 |           93381 |              26 |         gratuit
        u013563 |           93250 |              37 |         gratuit
        u025640 |           90301 |              31 |         gratuit
        u029858 |           90175 |              33 |         premium

--- Top 10 France / actif ---
        user_id |           score |       nb_events |            plan
        u048934 |           54994 |              25 |         premium
        u009679 |           54993 |              22 |         gratuit
        u015351 |           54982 |              24 |        standard
        u017027 |           54967 |              21 |        standard
        u020613 |           54964 |              24 |         gratuit
        u010041 |           54951 |              22 |        standard
        u045319 |           54948 |              22 |         gratuit
        u001195 |           54941 |              27 |         gratuit
        u044077 |           54938 |              23 |         gratuit
        u036149 |           54929 |              26 |        standard

--- Recherche globale par plan sur audience_par_pays_segment ---
CQL : SELECT user_id, country, segment, score FROM audience_par_pays_segment WHERE plan =
'premium' LIMIT 10
Refusee par Cassandra : Error from server: code=2200 [Invalid query] message="Cannot
execute this query as it might involve data filtering and thus may have unpredictable
performance. If you want to execute this query despite the performance unpredictability,
use ALLOW FILTERING"
Avec ALLOW FILTERING : 6352 lignes premium trouvees apres lecture des 34855 lignes de la
table (balayage complet)

Table audience_par_plan creee :
  PRIMARY KEY ((plan), score, user_id)
  CLUSTERING ORDER BY (score DESC, user_id ASC)
Lignes par plan :
  gratuit    16831  (attendu 16831)
  premium     6352  (attendu 6352)
  standard   11672  (attendu 11672)

--- Top 10 premium (audience_par_plan, une seule partition lue) ---
        user_id |           score |         country |         segment
        u041488 |          103073 |        Cameroun |        intensif
        u040743 |           96078 |   Cote_d_Ivoire |        intensif
        u049062 |           95407 |          Canada |        intensif
        u009818 |           95404 |         Senegal |        intensif
        u049503 |           92788 |          France |        intensif
        u025368 |           92400 |        Cameroun |        intensif
        u024406 |           92099 |        Belgique |        intensif
        u002854 |           90899 |         Senegal |        intensif
        u029858 |           90175 |          Canada |        intensif
        u004019 |           89320 |          Canada |        intensif
```

```
C3 - Requetes de service terminees.
```

La sortie complète de la partie A (Beeline, environ 2 000 lignes, plan EXPLAIN inclus) est dans `workspace/output/log_A.txt`.